

Social coding and open software - What can you do to get credit for your code and to allow reuse

In this lesson we will discuss how and why to share code and data, what kind of licenses are used in what situation, and how software can be cited.

We will try to connect software licenses to FAIR principles, give practical recommendations for starting, contributing, and reusing code and help you navigating and deciding on licenses.

📌 Why software licenses matter

- You **find some great code** or data that you want to reuse for your own publication (good for the original author: you will cite them and maybe other people who cite you will cite them).
- You need to **modify the code** a little bit, or you remix the data a bit.
- When it comes time to publish, you realize there is no **license**.

Now we have a problem:

- You manage to **publish the paper without the software/data** but others cannot build on your software and data and you don't get as many citations as you could.
- Or, you **cannot publish it at all** if the journal requires that papers should come with data and software so that they are reproducible.

This lesson is about how to avoid this situation for you and others.

⚙️ Prerequisites

No computational prerequisites required for this lesson.

20 min	Social coding
90 min	Software licensing focusing on open source
20 min	Software citation
10 min	Sharing data

Social coding

📌 Objectives

Question 1: Why would I want to share my scripts/code/data?

****Choose many****. Vote by adding an `o` character:

- A: Easier to find and reproduce (scientific reproducibility)
 - votes:
- B: More trustworthy: others can verify correctness and find and report bugs
 - votes:
- C: Enables others to build on top of your code (derivative work, provided the license allows it)
 - votes:
- D: Others can submit features/improvements
 - votes:
- E: Others can help fixing bugs
 - votes:
- F: Many tools and apps are free for open source, so no financial cost for this (GitHub, GitLab, Appveyor, Read the Docs)
 - votes:
- G: Good for your CV: you can show what you have built
 - votes:
- H: Discourages competitors. If others can't build on your work, they will make competing work
 - votes:
- I: When publicly shared, usually no time-stamp or set a version

Question 1: Why would I want to share my scripts/code/data?

****Choose many****. Vote by adding an `o` character:

- A: Easier to find and reproduce (scientific reproducibility)
- votes:
- B: More trustworthy: others can verify correctness and find and report bugs
- votes:
- C: Enables others to build on top of your code
(derivative work, provided the license allows it)
- votes:
- D: Others can submit features/improvements
- votes:
- E: Others can help fixing bugs
- votes:
- F: Many tools and apps are free for open source, so no financial cost for this
(GitHub, GitLab, Appveyor, Read the Docs)
- votes:
- G: Good for your CV: you can show what you have built
- votes:
- H: Discourages competitors. If others can't build on your work,
they will make competing work
- votes:
- I: When publicly shared, usually we time-stamp or set a version,
so it is easier to refer to a specific version
- votes:
- J: You can reuse your own code later after change of job or affiliation
- votes:
- K: It encourages me to code properly from the start
- votes:

Question 2: The most concerning thing for me, If I share my software now

****Choose one****. Vote by adding an `o` character:

- A: It will be scooped (stolen) by someone else
- votes:
- B: It will expose my "ugly code"
- votes:
- C: Others may find bugs and mistakes. What if the algorithm is wrong?
- votes:
- D: I will get too many questions, I do not have time for that
- votes:
- E: Losing control over the direction of the project
- votes:
- F: Low quality copies will appear
- votes:
- G: I won't be able to sell this later. Someone else will make money from it
- votes:
- H: It is too early, I am just prototyping, I will write version to distribute later
- votes:
- I: Worried about licensing and legal matters, as they are very complicated
- votes:

Question 3: Why is software often treated differently from papers?

Free-form answers:

- ...
- ...
- ...
- ...
- ...
- ...

Question 4: When you find a repository with code/library you would like to reuse, what are the things you look at to decide whether you use it?

Free-form answers:

- ...
- ...
- ...
- ...
- ...
- ...

```

- G: I won't be able to sell this later. Someone else will make money from it
  - votes:

- H: It is too early, I am just prototyping, I will write version to distribute later
  - votes:

- I: Worried about licensing and legal matters, as they are very complicated
  - votes:

```

Question 3: Why is software often treated differently from papers?

Free-form answers:

```

- ...
- ...
- ...
- ...
- ...
- ...

```

Question 4: When you find a repository with code/library you would like to reuse, what are the things you look at to decide whether you use it?

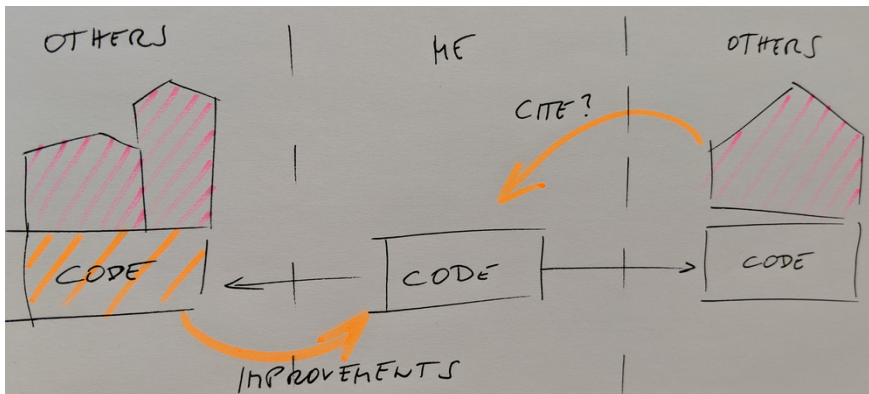
Free-form answers:

```

- ...
- ...
- ...
- ...
- ...
- ...

```

- The goal is maximum visibility and maximum reuse.
- The more interesting science is done referencing my paper, the better for me.
- Nobody actively tries to limit the reach of their papers.



Different ways we can benefit from sharing code.

Sharing code:

- “I did all the ground work and they get to do the interesting science?”
- Sharing code and encouraging *derivative work* may boost your academic impact.

However a study showed that despite these policies, many people still do not share their code 😞. This paper includes samples of charming author responses such as:

Journal policies as motivation for sharing

“When you approach a PI for the source codes and raw data, you better explain who you are, whom you work for, why you need the data and what you are going to do with it.”
From Science editorial policy

Motivation for open source software

“We require that all computer code used for modeling and/or data analysis that is not

Instructor note available be deposited in a publicly accessible repository upon publication.

In rare exceptional cases where security concerns or competing commercial interests pose a conflict, code-sharing arrangements that still facilitate reproduction of the work should be discussed with your Editor no later than the revision stage.”

- Enable derivative work

Do not lock yourself out of own code

- Attract developers who want to be able to show the coding work on their CVs
- Tighter regulated domains (e.g. medicine) researchers should be able to replicate and build upon the work of others (PSS) can lead to more engagement from industry. Research may lead to journals that authors are required to make **materials, data, code, and associated protocols** promptly available to readers without undue qualifications. Any restrictions on the availability of materials or information must be disclosed to the editors at the time of

Sharing software is also scary so be disclosed in the submitted manuscript.”

Instructor note

How will your work be visible if it is used by others, many people?

However, a study showed that despite these policies, many people still do not share their code 😞. This paper includes samples of charming author responses such as:

Journal policies as motivation for sharing

"When you approach a PI for the source codes and raw data, you better explain who you are, whom you work for, why you need the data and what you are going to do with it."

Motivation for open source software

"We require that all computer code used for modeling and/or data analysis that is not

Instructor note available be deposited in a publicly accessible repository upon publication.

In rare exceptional cases where security concerns or competing commercial interests pose a conflict, code-sharing arrangements that still facilitate reproduction of the work should be discussed with your Editor no later than the revision stage."

- Enable derivative work

Do not lock yourself out of own code

- Attract developers who want to be able to show the coding work on their CVs
- Tight regulation of domain publication open source
- Open the code software (OSS) can lead to more engagement from industry, which may lead to more impact
- Journal authors are required to make **materials, data, code, and associated protocols** promptly available to become standard
- Indue qualifications. Any restrictions on the availability of materials or information must be disclosed to the editors at the time of

Sharing software is also scary

Instructor note

We revisit answers to Question 2 (above).

- **Fear of being scooped:** A license can avoid it, and you can release when you are ready. Anyway, it is very unlikely that others will understand your code and publish before you without involving you in a collaboration. Sharing is a form of publishing.
- **Exposes possibly "ugly code":** In practice almost nobody will judge the quality of your code. "Software, once written, is never really finished" (N. Asparouhova).
- **Others may find bugs and mistakes:** Isn't this good? Would you not like to use a code which gives people the chance to locate bugs? If you don't release, people will assume there are bugs anyway.
- **Others may require support and ask too many questions:** This can become a problem: use tools and community and protect your time. You aren't required to support anyone. You can also "archive" a repository to disable most forms of interaction (issues, PRs). Also a note in README on support level helps.
- **Fear of losing control over the direction of the project:** Open source does not mean everybody can change **your version**.
- **"Bad" derivative projects may appear:** It will be clear which is the official version.

Code reusability: What contributes to reusability?

What contributes to you being able to reuse stuff that others make, and others (or you) being able to reuse your stuff? When you find a repository with code you would like to reuse, you may look at the following things to determine its reusability:

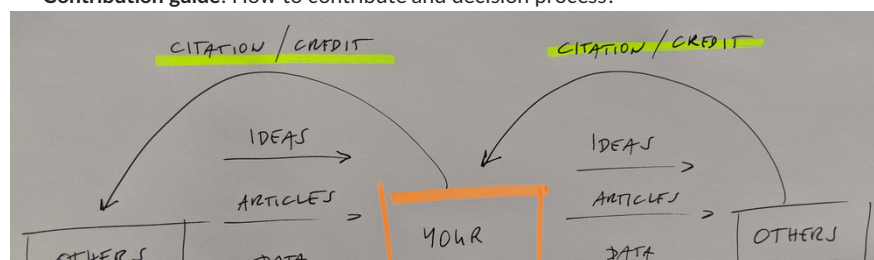
- Have you experienced an implicit expectation of support?
- Supporting all requests can lead to overworking and mental health issues.
- Not supporting requests can also induce guilt.
- Most projects are maintained by 1 or 2 persons.

This can be viewed in connection to Question 4 (above). Most projects connected to Question 4 (above) are longer time. Interests change. "Casual contributors are like tourists visiting NYC for a weekend" (Nadia Asparouhova, book below).

- **Date of last code change:** Is the project abandoned?
- If you maintain all projects that you start forever, at some point it may be difficult to start new projects.
- **Release history:** How about stability and backwards-compatibility?
- **Versioning:** Will it be painful to upgrade?
- What are your experiences? Do you agree with the above thoughts?
- **Number of open pull requests and issues:** Are they followed-up?
- Book recommendation: Nadia Asparouhova (formerly Nadia Eghbal): "Working in Public: The Making and Maintenance of Open Source Software (Stripe Press)"
- **Example:** Will it be difficult to get started?

How our work connects to the work of others

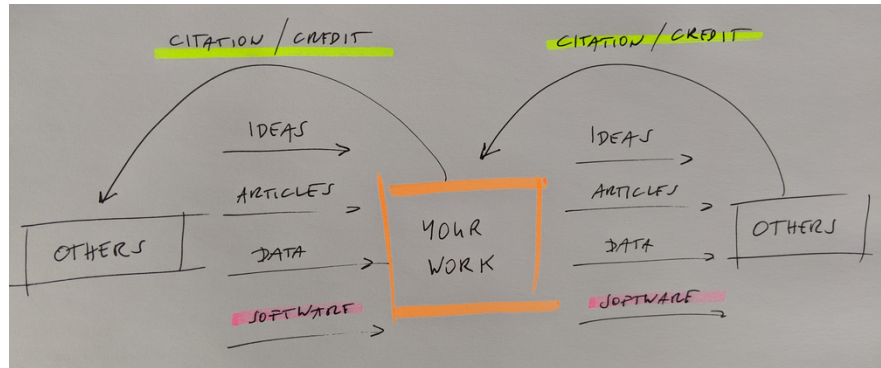
- **Contribution guide:** How to contribute and decision process?



- Have you experienced an implicit expectation of support? You may look at the following things to determine its reusability:
 - Supporting all requests can lead to overworking and mental health issues.
 - Not supporting requests can also induce guilt.
 - Most projects are maintained by 1 or 2 persons.
- This is not a project, it's a community. Contributors for a longer time. Interests change. "Casual contributors are like tourists visiting NYC for a weekend" (Nadia Asparouhova, book below).
- **Date of last code change:** Is the project abandoned?
 - If you maintain all projects that you start forever, at some point it may be difficult to start new projects.
 - **Release history:** How about stability and backwards-compatibility?
 - **Versioning:** Will it be painful to upgrade?
 - What are your experiences? Do you agree with the above thoughts?
 - **Number of open pull requests and issues:** Are they followed-up?
 - Book recommendation: Nadia Asparouhova (formerly Nadia Eghbal): "Working in Public: The Making and Maintenance of Open Source Software (Stripe Press)"
 - **Installation instructions:** Will it be difficult to get it running?
 - **Example:** Will it be difficult to get started?

How our work connects to the work of others

- **Contribution guide:** How to contribute and decision process?



Whether and what we can share depends on how we obtained the components.

- Our work depends on outputs from others. Research of others depends on our outputs.
- Whether you can share your output depends on how you obtained your input.
- A repository that is private today might become public one day.
- Sometimes "OTHERS" are you yourself in the future in a different group/job.
- Our code often starts by changing other code: **derivative work**.
- **Software licenses** matter. And this is what we will discuss in the next episode.

Software licensing focusing on open source

Objectives

- Principles of open source licensing
- Difference between permissive and copyleft licenses
- Regulations for AI-generated and AI-assisted code
- Determine the software license for your project following EU regulation
- Navigate the Joinup Licensing Assistant to select a compliant license
- Understand the licensing distinction between container recipes and container images

Research Software Alliance Policy Directory

This lesson is designed as practical educational material for researchers and research

Introduction: What is a Software License?

Under copyright law, worldwide software without an explicit license automatically defaults to "All Rights Reserved", meaning nobody else has the legal right to reuse, modify, embed, or cite your code. A software license is a legal permission grant created by the author that overrides this statutory default, defining how downstream researchers can reuse your work. Policies heavily underlie software ownership and licensing choices.

- This lesson covers only the general principles of open-source reuse, copyright scope, and software classification.
- Open-source licenses fall into two main families:
 - **Permissive (e.g., MIT, Apache-2.0, BSD):** "Do whatever you want, just keep credit." Grants maximum reuse freedom, allowing anyone to modify, embed, or relicense your code in open or closed projects.
 - **Copyleft / Reciprocal (e.g., GPL-3.0, EUPL-1.2):** "Share alike." Grants full freedom to run and modify, but mandates that any distributed derivative work must also be released under the same open-source copyleft terms.

If you need formal guidance references below and legal experts, especially if you have legal services at your host institution, could be of help:

- [EU Directive 2009/24/EC](#)
- [Compendium of U.S. Copyright Office Practices \(3rd Ed.\) – Chapter 700, Section 721: Computer Programs](#)

The diagram below unifies these license choices and their downstream rights:

- [Chinese Regulations on Computer Software Protection](#)(search:"????????")
- [Joinup Licensing Assistant](#) Variables: { 'edgeLabelBackground': '#ffffff' } }%
- [FSF REUSE Initiative](#)

Flowchart TD
 A["Your Research Codebase
<i>(Source code, container recipes, prompt templates)</i>"] -->|No Lice

This lesson is designed as practical educational material for researchers and research

Introduction: What is a Software License?

Introduction: What is a Software License?

Under copyright law, worldwide, software without an explicit license automatically defaults to “All Rights Reserved”, meaning nobody else has the legal right to run, modify, embed, or cite your code. A software license is a legal permission grant created by the author that overrides this statutory default, defining how downstream researchers can reuse your work. Policies heavily influence software ownership and licensing choices.

- This lesson covers only the general principles of open-source reuse, copyright scope, and software adaptation!
- Open-source licenses fall into two main families:
 - Permissive** (e.g., MIT, Apache-2.0, 0BSD): “Do whatever you want, just keep credit.”

Grants maximum reuse freedom, allowing anyone to modify, embed, or relicense your code in open or closed product.

- If you need more guidance, references below are good experts, especially if you have legal services at your host institution, could be of help:
- Copyleft / Reciprocal** (e.g., GPL-3.0, EUPL-1.2): “Share alike.” Grants full freedom to run and modify, but mandates that any distributed derivative work must also be released under the same open-source copyleft terms.
- EUR Directive 2009/24/EC
- Compendium of U.S. Copyright Office Practices (3rd Ed.) – Chapter 700, Section 721:

[Computer Programs](#)

The diagram below unifies these license choices and their downstream rights:

- [Chinese Regulations on Computer Software Protection](#)(search:"????????")

- [Joinup%{adding As theme Variables': { 'edgeLabelBackground': '#ffffff' }}}%](#)

flowchart TD

[FSF REUSE Initiative](#)

Al"Your Research Codebase
<i>(Source code, container recipes, prompt templates)</i>"] -->|No Lice

A -->|Attach Open-Source License
(Explicit Permission Grant)"| C{"Select License Flavor"}

C -->|Permissive
(MIT, Apache-2.0, 0BSD)"| D["Permissive License"]

C -->|Copyleft / Reciprocal
(GPL-3.0, EUPL-1.2)"| E["Copyleft License"]

C -->|Proprietary / Closed Source"| F["Closed Source / Restricted
🚫 <i>Flavour not discussed in

D --> D1["Run & Modify? Yes!"]

D --> D2["Embed in closed product? Yes!"]

D --> D3["Must changes stay open? No (Optional)"]

E --> E1["Run & Modify? Yes!"]

E --> E2["Embed in closed product? No!"]

E --> E3["Must changes stay open? Yes! (Mandatory)"]

classDef defaultState fill:#ffe3e3,stroke:#e03131,stroke-width:2px,color:#5c0000;

classDef openState fill:#e6ffe6,stroke:#2b8a3e,stroke-width:2px,color:#1b4332;

classDef copyleftState fill:#fff9db,stroke:#f59f00,stroke-width:2px,color:#5c3c00;

classDef closedState fill:#f8f9fa,stroke:#adb5bd,stroke-width:2px,stroke-dasharray: 5 5,color:#6c757d;

classDef codeState fill:#f8f9fa,stroke:#495057,stroke-width:2px,color:#212529;

class B,E2 defaultState;

class D,D1,D2,D3,E1 openState;

class E,E3 copyleftState;

class F closedState;

class A,C codeState;

Across international legal frameworks (such as 17 U.S.C. § 101 and WIPO model provisions),

Copyright Foundation: Expression vs. Ideas

Copyright protects the **expression** of an idea, which can be used directly or indirectly in a computer to bring about a certain result.

To understand why licenses are required, you must understand how copyright law treats

software under EU statutory law (Directive 2009/24/EC) and international treaties. Software and licensing principles for research software assets:

How Copyright Original draws a sharp fundamental distinction between what is protected and what is free for anyone to use:

- Third-Party Integrations:** Embedded permissive or copyleft code snippets and
- Protected (Code Expression):** The specific source code text, variable names, binaries,
- Infrastructure Code:** Any code engineering, tool, and infrastructure design, like Recipes
- (Dockerfile , A Dockerfile .def , as):** Mathematical algorithms, scientific models,
- Container Images:** Bundled binaries, system snapshots ('.sif' files, OCI registry images).
- AI-Assisted Code:** Code generated, refactored, or assembled with human creative

Because copyright restricts only the *creative human expression* and not the underlying *ideas* or

algorithms, developers use open-source licenses to define the exact terms under which that expression can be legally shared and modified, meeting the threshold of human creative authorship.

Scope of this Lesson: What Counts as “Software”?

Notation: Debugging a License Compliance Failure

With the understanding of the difference between Permissive (MIT) and Copyleft (GPL-3.0)

license, examine what happens when they collide inside an automated CI/CD pipeline:

Copyright Foundation: Expression vs. Ideas

To understand why licenses are required, you must understand how copyright law treats

However, Copyright law draws a sharp, fundamental distinction between what is protected and what is free for anyone to use:

- **Third-Party Integrations:** Embedded permissive or copyleft code snippets and
 - **Protected Code Expression:** The specific source code text, variable names, binaries, and infrastructure code. Prohibit playgrounds, IDEs, and configuration design files (e.g., `Dockerfile`, `Makefile`, `def` files).
 - **Mathematical Algorithms:** Mathematical algorithms, scientific models, and engineering designs. Data structures and interface principles (e.g., `.cif` files, OCI registry images).
 - **AI-Assisted Code:** Code generated, refactored, or assembled with human creative oversight.
- Because copyright restricts only the *creative human expression* and not the underlying *ideas or algorithms*, developers use open-source licenses to define the exact terms under which that **AI Prompt Templates** complex, engineered system prompts and structured frameworks expression can be legally shared and modified, meeting the threshold of human creative authorship.

Scope of this Lesson: What Counts as “Software”?

```

%%{init: {'themeVariables': { 'edgeLabelBackground': '#faf5ff' }}}%%
flowchart TB
    subgraph box["CI/CD License Compliance Debugging Pipeline"]
        A["<b>Build Trigger:</b> Push to my-analysis-tool"] --> B["Run Compliance Scanner"]
        B --> C["Check Inbound vs.<br/>Outbound Terms"]

        C -->| "Your Target License: MIT (Permissive)<br/>Pasted Snippet: GPL-3.0 (Copyleft)" | D["❌ <b>BUILD FAIL"]

        D --> E["Select Patch Option"]

        E -->| "Option A: Keep MIT & add comment '# Originally GPL'" | F["❌ <b>BUILD FAIL</b><br/>Comments do not"]
        E -->| "Option B: Re-license repo to GPL-3.0 / EUPL-1.2" | G["✅ <b>BUILD PASS</b><br/>Your license matches"]
        E -->| "Option C: Rewrite code from scratch to replace snippet" | H["✅ <b>BUILD PASS</b><br/>New code expr"]
        E -->| "Option D: Delete LICENSE file to bypass scanner" | I["⚠️ <b>PASSED SCANNER (TRAP!)</b><br/>No licen"]

        P["<b>Permissive</b><br/>(MIT, Apache-2.0, 0BSD)<br/><i>'Do whatever you want, just keep credit'</i>"]
        CL["<b>Copyleft / Reciprocal</b><br/>(GPL-3.0, EUPL-1.2)<br/><i>'Must share changes under same terms'</i>"]
    end

    P -.->| "I want to use" | C
    CL -.->| "Pasted code snippet uses" | C

    classDef pass fill:#e6ffe6,stroke:#2b8a3e,stroke-width:2px,color:#1b4332;
    classDef copyleft fill:#fff9db,stroke:#f59f00,stroke-width:2px,color:#5c3c00;
    classDef fail fill:#ffe3e3,stroke:#e03131,stroke-width:2px,color:#5c0000;
    classDef warning fill:#fff3bf,stroke:#f08c00,stroke-width:2px,color:#5c0000;
    classDef neutral fill:#f8f9fa,stroke:#495057,stroke-width:2px,color:#212529;
    classDef flowchart fill:#ffffff,stroke:#adb5bd,stroke-width:1px;

    class box flowchart;
    subgraph box [ ]
        class A,B,C,D,E,F,G,H,I,P,CL pass;
        class J["Public Rights Law Foundation<br/>(EU Directive 2009/24/EC)"] --> B["Permissive<br/>(MIT, BSD, Apache-2.0)"]
        class K["Copyleft / Reciprocal<br/>(EUPL, GPL, LGPL)"]
        class L["FDFA Rights Reserved / Proprietary"]
        class I warning;
        class M["Embed & Modify?<br/><b>Yes!</b>"]
        class N["Embed & Sell copies as-is?<br/><b>Yes!</b>"]
        class O["Embed in closed product & sell?<br/><b>Yes!</b>"]
        class P["Must changes stay open?<br/><b>No</b> (Optional)"]

        C --> C1["Run & Modify?<br/><b>Yes!</b>"]
        C --> C2["Sell copies as-is?<br/><b>Yes!</b>"]
        C --> C3["Embed in closed product & sell?<br/><b>No!</b>"]
        C --> C4["Must changes stay open?<br/><b>Yes!</b> (Mandatory)"]

        D --> D1["Run & Modify?<br/><b>No!</b> (Zero permission)"]
        D --> D2["Sell copies as-is?<br/><b>No!</b>"]
        D --> D3["Embed in closed product & sell?<br/><b>No!</b>"]
        D --> D4["Can I change code?<br/><b>No</b> (Closed source)"]
    end
    subgraph box ["OSI compatible"]
        B["Permissive<br/>(MIT, BSD, Apache-2.0)"]
        C["Copyleft / Reciprocal<br/>(EUPL, GPL, LGPL)"]
    end
end
end

```

Classification of licenses

```
classDef fill fill:#f0f0f0,stroke:#000,stroke-width:1px,color:#000;
classDef warning fill:#fff3bf,stroke:#f08c00,stroke-width:2px,color:#5c0000;
classDef neutral fill:#f8f9fa,stroke:#495057,stroke-width:2px,color:#212529;
classDef box fill:#ffffff,stroke:#adb5bd,stroke-width:1px;

subgraph box[ ]
class B1,B2,B3,B4,C1,C2,C3,C4["Permissive<br/>(MIT, BSD, Apache-2.0)"]
class C1,C2,C3,C4["Copyleft / Reciprocal<br/>(EUPL, GPL, LGPL)"]
class D1,D2,D3,D4["All Rights Reserved / Proprietary"]
class I warning;
class A,B,C,D["Run & Modify?<br/><b>Yes!</b>"]
class B1,B2,B3,B4["Sell copies as-is?<br/><b>Yes!</b>"]
B --> B3["Embed in closed product & sell?<br/><b>Yes!</b>"]
B --> B4["Must changes stay open?<br/><b>No</b> (Optional)"]

C --> C1["Run & Modify?<br/><b>Yes!</b>"]
C --> C2["Sell copies as-is?<br/><b>Yes!</b>"]
C --> C3["Embed in closed product & sell?<br/><b>No!</b>"]
C --> C4["Must changes stay open?<br/><b>Yes!</b> (Mandatory)"]

D --> D1["Run & Modify?<br/><b>No!</b> (Zero permission)"]
D --> D2["Sell copies as-is?<br/><b>No!</b>"]
D --> D3["Embed in closed product & sell?<br/><b>No!</b>"]
D --> D4["Can I change code?<br/><b>No</b> (Closed source)"]
subgraph osi["OSI compatible"]
B["Permissive<br/>(MIT, BSD, Apache-2.0)"]
C["Copyleft / Reciprocal<br/>(EUPL, GPL, LGPL)"]
end
end

classDef permissive fill:#e6ffe6,stroke:#2b8a3e,stroke-width:2px,color:#1b4332;
classDef copyleft fill:#fff9db,stroke:#f59f00,stroke-width:2px,color:#5c3c00;
classDef proprietary fill:#ffe3e3,stroke:#e03131,stroke-width:2px,color:#5c0000;
classDef header fill:#f8f9fa,stroke:#495057,stroke-width:2px,color:#212529;
classDef mains fill:#fafadc,stroke:#495057,stroke-width:2px,color:#212529;
classDef osiBox fill:#f8f9fa,stroke:#0275d8,stroke-width:2px,stroke-dasharray: 5 5,color:#0275d8;
classDef box fill:#ffffff;
class B1,B2,B3,B4,C1,C2 permissive;
class C3,C4,D1,D2,D3,D4 proprietary;
class box box;
class A,B,C,D mains;
class osi osiBox;
```

Selecting Compliant Licenses

When using the European Commission's [Joinup Licensing Assistant \(JLA\)](#), license selection depends on your RSE workflow. The JLA groups criteria into four categories: **Can** (Permissions), **Must** (Obligations), **Compatible** (Domain), and **Support** (OSI Approval).

JLA Decision Matrix at a Glance

Scenario Module	Key JLA Toggle (Must)	Resulting Category	Target Licenses
Global Context: Software Engineering Across Legal Borders			
1. Own Code	Incl. Copyright	Permissive	MIT , Apache-2.0 , BSD-3-Clause
Software development is inherently cosmopolitan: research software engineers routinely collaborate across legal borders, fetch dependencies from global registries, and commit code to international repositories.			
2. Multi-Implementation	Copyright / Patent Disclose Source	Copyleft	EUPL-1.2 , GPL-3.0 , AGPL-3.0
3. Embed Permissive	Incl. Copyright	Flexible (Any)	MIT or EUPL-1.2 / GPL-3.0
However, modern developers face a subtle trap: AI legal bias . Coding assistants (ChatGPT, Claude, GitHub Copilot) are overwhelmingly trained on US-centric web data and legal texts.			
4. Embed Copyleft	Copyright / Patent (Mandatory)	Copyleft	EUPL-1.2 , GPL-3.0
Consequently, when asked about software ownership or licensing, AI outputs almost universally default to US common law concepts ("Fair Use", "Work Made for Hire", "Derivative Works"). Relying blindly on AI advice can create legal blind spots when operating under EU statutory frameworks or collaborating globally.			
5. Link or Library	Copyright / Patent Share a	Copyleft	EUPL-1.2 , GPL-3.0 , AGPL-3.0
6. AI-Assisted Code	Incl. Copyright	Author Choice	MIT , Apache-2.0 (or Copyleft)
7. Container	Incl. Copyright	Permissive	MIT , Apache-2.0
Selecting Compliant Licenses			
8. Built Image	Overlapping Component	Multi-License	Governed by individual image layers
When using the European Commission's Joinup Licensing Assistant (JLA) , license selection depends on your RSE workflow. The JLA groups criteria into four categories: Can (Permissions), Must (Obligations), Compatible (Domain), and Support (OSI Approval).			
9. Prompt Template	Incl. Copyright	Permissive	MIT , Apache-2.0

JLA Decision Matrix at a Glance

Global Context: Software Engineering Across Legal Borders

1. Own Code	Incl. Copyright	Permissive	MIT , Apache-2.0 , BSD-3-Clause
2. Math Implementation	Copyleft/Share a. + Disclose Source	Copyleft	EUPL-1.2 , GPL-3.0 , AGPL-3.0
3. Embed Permissive	Incl. Copyright	Flexible (Any)	MIT or EUPL-1.2 / GPL-3.0
4. Embed Copyleft	Copyleft/Share a. (Mandatory)	Copyleft	EUPL-1.2 , GPL-3.0
5. Link GPL Library	Copyleft/Share a. (Mandatory)	Copyleft	GPL-3.0 , EUPL-1.2
6. AI-Assisted Code	Incl. Copyright	Author Choice	MIT , Apache-2.0 (or Copyleft)
7. Container Recipe	Incl. Copyright	Permissive	MIT , Apache-2.0
8. Built Image	Overlapping Component Terms	Multi-License	Governed by individual image layers
9. Prompt Template Approval	Incl. Copyright	Permissive	MIT , Apache-2.0

Software development is inherently cosmopolitan: research software engineers routinely collaborate across legal borders, fetch dependencies from global registries, and commit code to international repositories.

However, modern developers face a subtle trap: **AI legal bias**. Coding assistants (ChatGPT, Claude, GitHub Copilot) are overwhelmingly trained on US-centric web data and legal texts. Consequently, when asked about software ownership or licensing, AI outputs almost universally default to **US common law concepts** (“Fair Use”, “Work Made for Hire”, “Derivative Works”). Relying blindly on AI advice can create legal blind spots when operating under EU statutory frameworks or collaborating globally.

When using the European Commission’s [Joinup Licensing Assistant \(JLA\)](#), license selection depends on your RSE workflow. The JLA groups criteria into four categories: **Can** (Permissions), **Must** (Obligations), **Compatible** (Domain), and **Support** (QSL Template Approval).

Selecting Compliant Licenses

JLA Decision Matrix at a Glance

Scenario Module	Key JLA Toggle (☐ Must)	Resulting Category	Target Licenses
1. Own Code	Incl. Copyright	Permissive	MIT , Apache-2.0 , BSD-3-Clause
2. Math Implementation	Copyleft/Share a. + Disclose Source	Copyleft	EUPL-1.2 , GPL-3.0 , AGPL-3.0
3. Embed Permissive	Incl. Copyright	Flexible (Any)	MIT or EUPL-1.2 / GPL-3.0
4. Embed Copyleft	Copyleft/Share a. (Mandatory)	Copyleft	EUPL-1.2 , GPL-3.0
5. Link GPL Library	Copyleft/Share a. (Mandatory)	Copyleft	GPL-3.0 , EUPL-1.2
6. Container Recipe	Incl. Copyright	Permissive	MIT , Apache-2.0
7. Built Image	Overlapping Component Terms	Multi-License	Governed by individual image layers
8. AI-Assisted Code	Incl. Copyright	Author Choice	MIT , Apache-2.0 (or Copyleft)
9. Prompt Template	Incl. Copyright	Permissive	MIT , Apache-2.0

- **Outcome: Fully Permissible.** You own the code and can choose any open-source license.

Module 1: Clean Slate – Authoring Original Code & Algorithms

- **Selected Category: Permissive** (driven by your goal of maximum adoption).

When **authoring original code** (e.g., mathematical logic), you control 100% of your work.

- **User Obligation:** Downstream users must comply with individual external package licenses when fetching or running dependencies.
- 👉 **Public Domain vs. Permissive Licenses:** Civil law jurisdictions (EU, China, Japan, South Korea) do not recognize total waivers of moral rights (e.g., your right to attribution as an author). Avoid informal “Public Domain” claims; always use standard permissive open-source licenses (MIT , 0BSD , Apache-2.0) to grant legal permissions safely worldwide.
- **Licensing Goal:** You want **maximum adoption** and zero friction for commercial or academic reuse.

Scenario 2: Implementing an algorithm from a paper

- **JLA Filter Focus:** Select ☒ Commercial use , ☒ Modify , ☒ Distribute , ☐ Incl.

You read a published scientific paper, understand the underlying mathematical algorithm, and write your own original software implementation from scratch.

✓ **Solution**

- **Licensing Goal:** You want **reciprocal protection**—anyone can use your code, but downstream modifications distributed by others must remain open source.
- **Legal Reality:** External dependencies remain separate works. Because you have not bundled third-party code inside your repository, those dependencies must remain open source.

JLA Filter Focus: Add ☐ Must toggle: Copyleft/Share a. , ☒ Disclose source , or ☐ original codebase.

✓ **Solution**

- **Outcome:** **Fully Permissible.** You own the code and can choose any open-source license.

Module 1: Clean Slate – Authoring Original Code & Algorithms

- **Selected Category:** **Permissive** (driven by your goal of maximum adoption).

When **authoring original code**, you control 100% of your repository.

User Obligation: Downstream users must comply with individual external package licenses when fetching or running dependencies.

👉 • **Public Domain vs. Permissive Licenses:** Civil law jurisdictions (EU, China, Japan, South Korea) do not recognize total waivers of moral rights (e.g., your right to attribution as an author). Avoid informal “Public Domain” claims; always use standard permissive open-source licenses (`MIT` , `0BSD` , `Apache-2.0`) to grant legal permissions safely worldwide. (`requirements.txt` , `CMakeLists.txt` , `Cargo.toml`).

- **Licensing Goal:** You want **maximum adoption** and zero friction for commercial or academic reuse.

👉 **Scenario 2: Implementing an algorithm from a paper**

JLA Filter Focus: Select ☒ `Commercial use` , ☒ `Modify` , ☒ `Distribute` + ☐ `Incl.`

You read a published scientific paper, understand the underlying mathematical algorithm, and write your own original software implementation from scratch.

✓ **Solution**

- **Licensing Goal:** You want **reciprocal protection**—anyone can use your code, but downstream modifications distributed by others must remain open source.

Legal Reality: External dependencies remain separate works. Because you have not bundled third-party code inside your repository, you must remain open source.

👉 • **JLA Filter Focus:** Add ☐ `Must` toggle. `Copyright/Share a.` , `llc` , `Disclose source` , `or` original codebase.

✓ **Solution**

Legal Reality: Under EU Directive 2009/24/EC Art. 1(2), copyright protects specific source code *expression*, not underlying mathematical algorithms or scientific principles. Writing a fresh implementation creates a brand-new copyright.

- **Outcome:** **Fully Permissible.** You own 100% of the copyright for your software implementation.
- **Selected Category:** **Copyleft / Reciprocal** (driven by your goal of community protection).
- **JLA Matches:** `EUPL-1.2` , `GPL-3.0` , `AGPL-3.0`
- **User Obligation:** Users who redistribute your software or their modified versions must provide source code access under matching copyleft terms.

Module 2: The Dependency Minefield – Inbound Code & Linking

Embedding third-party source code snippets or linking against strong copyleft libraries introduces legal boundaries that restrict your repository choices.

👉 **Scenario 3: Directly embedding third-party Permissive source code**

You copy and paste a helper module licensed under a **Permissive license** (e.g., MIT or 0BSD) and paste a utility function licensed under a **Copyleft / Reciprocal license** (e.g., GPL-3.0 or EUPL-1.2) directly into your repository.

- **Licensing Goal:** Know if including permissive third-party code limits your overall repository license choices.

JLA Filter Focus: Base case ☒ `Commercial use` + ☐ `Incl. Copyright` (Permissive code leaves all target options open).

👉 • **JLA Filter Focus:** `Must` clause `Copyleft/Share a.` is **mandated** by inbound code.

✓ **Solution**

Legal Reality: Parting third-party copyleft source code directly into your repository creates a single combined work. You do not hold exclusive copyright over the overall codebase, provided you preserve the original author's copyright notice.

- **Outcome:** **Full Flexibility.** Embedding Permissive code does not force a specific license on your project or create derivative copyleft.
- **EU vs. US Legal Concepts (Adaptation vs. Derivative Work):** Coding AI tools often refer to this under the US common-law doctrine of “Derivative Works”. In the EU (Directive 2009/24/EC Art. 4(1)(b)), modifying or refactoring code is classified as a statutory act of **Adaptation, Translation, or Alteration**. Regardless of terminology, modifying copyleft code triggers mandatory reciprocal sharing obligations.
- **Outcome:** **Restricted Choice (Mandatory Copyleft).** You cannot choose a permissive license (MIT) or keep the repository proprietary.

👉 • **Selected Category:** **Copyleft / Reciprocal**

- **JLA Matches:** `EUPL-1.2` , `GPL-3.0`
- **User Obligation:** Anyone distributing your project must provide access to the full

You copy and paste a utility function licensed under a **Copyleft / Reciprocal license** (e.g., GPL-3.0 or EUPL-1.2) directly into your repository files.

- **Licensing Goal:** Know if including permissive third-party code limits your overall
- **Licensing Goal:** Fulfill legal obligations imposed by incorporating inbound copyleft
- **JLA Filter Focus:**  **Commercial use** +  **Incl. Copyright** (Permissive code
- **JLA Filter Focus:**  **Must clause** **Copyleft/Share a.** is **mandated** by inbound code. (leaves all target options open).

✔ **Solution**

Legal Reality: Permissive licenses grant broad rights to combine, modify, and re-license even derivative works, provided you preserve the original author's copyright notice. You do not hold exclusive copyright over the overall codebase.

- **Outcome: Full Flexibility.** Embedding Permissive code does not force a specific
- **EU vs. US Legal Concepts (Adaptation vs. Derivative Work):** Coding AI tools often
- **JLA Matches:** **EUPL-1.2**, **GPL-3.0**, **MIT**, **Apache-2.0** (Derivative Works". In the EU
- **User Obligation:** Retain the original copyright notice and MIT/BSD license text as a statutory act of **Adaptation, Translation, or Alteration**. Regardless of terminology, modifying copyleft code triggers mandatory reciprocal sharing obligations.
- **Outcome: Restricted Choice (Mandatory Copyleft).** You cannot choose a
-  **permissive license (MIT) or keep the repository proprietary.**
- **Selected Category: Copyleft / Reciprocal**
- **JLA Matches:** **EUPL-1.2**, **GPL-3.0**
- **User Obligation:** Anyone distributing your project must provide access to the full source code under matching copyleft terms.

 Scenario 5: Linking against a Strong Copyleft library (e.g., GSL or FFTW)

You write your code from scratch, but your program links (statically or dynamically) against a scientific library licensed under **GPL-3.0**.

- **Licensing Goal:** Select a license compliant with the inbound linking requirements of the GPL library.
- **JLA Filter Focus:**  **Must clause** **Copyleft/Share a.** + **Disclose source** (Required across linking boundaries).

✔ **Solution**

Legal Reality: Linking your code with a Strong Copyleft library like GPL creates a combined software work upon compilation and distribution.

- **Outcome: Mandatory Copyleft.** To distribute the compiled application or repository, your code must be licensed under a GPL-compatible copyleft license.
- **JLA Matches:** **GPL-3.0**, **AGPL-3.0**, **EUPL-1.2**
- **User Obligation:** Anyone distributing compiled binaries must provide the full application source code under GPL-compatible copyleft terms.
- **Outcome: Fully Permissible.** You own the copyright to the build instructions and can choose any license for your recipe file.
- **JLA Matches:** **MIT**, **Apache-2.0**, **BSD-3-Clause**

Module 9: Reproducible Infrastructure - Build Recipes vs. Binary Bundles

A major trap for RSEs is confusing **Infrastructure as Code** text files (recipes) with **compiled binary filesystems** (container images).

 Scenario 7: Distributing a Built Container Image (Docker Hub or Apptainer .sif)

 **Scenario 6: Distributing a Container Build Recipe (Dockerfile or Apptainer .def)**
You build and publish a complete container runtime image (**.sif** or Docker Hub image) **without** a base container OS/distro libraries dependencies and your application code. You write commands that pull base images and install packages.

- **Licensing Goal:** Comply with legal obligations when distributing a bundled binary
- **Licensing Goal:** Maximum adoption for your build instructions with zero restrictions.
- **JLA Filter Focus:** **MIT** (Getting around a single **JLA** license filter to a multi-work binary bundle).

✔ **Solution**

Legal Reality: Unlike a text recipe file, a compiled container image is a **bundle of separate third-party software works**. You do not hold exclusive copyright over the entire image filesystem (infrastructure as Code). Referencing external base images or packages in commands does not transfer third-party copyright onto your text file.

- **Outcome: Fully Permissible.** You own the copyright to the build instructions and can choose any license for your recipe file.
- **JLA Matches:** MIT , Apache-2.0 , BSD-3-Clause

Module 3: Reproducible Infrastructure – Build Recipes vs. Binary Bundles

A major trap for RSEs is confusing **Infrastructure as Code** text files (recipes) with **compiled binary filesystems** (container images).

🔧 Scenario 7: Distributing a Built Container Image (Docker Hub or Apptainer .sif)

🔧 Scenario 6: Distributing a Container Build Recipe (Dockerfile or Apptainer .def)

You build and publish a complete container runtime image (`.sif` or Docker Hub image) **you direct a base image OS, system libraries, dependencies, and your application** in text commands that pull base images and install packages.

- **Licensing Goal:** Comply with legal obligations when distributing a bundled binary
- **Licensing Goal:** Maximum adoption for your build instructions with zero restrictions.
- **JLA Filter Focus:** Treat (or opt into) a single JLA license filter to a multi-work binary bundle) .

✓ Solution

Legal Reality: Unlike a text recipe file, a compiled container image is a **bundle of separate third-party software works**. You do not hold exclusive copyright over the entire image filesystem. Referencing external base images or packages in commands does not transfer third-party copyright onto your text file.

- **Outcome: Mandatory Multi-License Compliance.** Distribution is governed by the overlapping terms of all installed base layers, packages, and linked binaries inside.
- **Key Rule:** If your application links against a GPL library inside the container, image distribution triggers GPL source disclosure obligations for your app. If GPL tools in the container are standalone system utilities, “mere aggregation” applies.
- **User Obligation:** Ensure compliance with all third-party licenses bundled inside the container layers.

Module 4: Modern AI Workflows – Assisted Code & Prompt Engineering

AI tools introduce distinct licensing considerations depending on whether you integrate AI-generated code snippets or author complex system prompt templates.

🔧 Scenario 8: Generating or assisting code using AI tools

You write software using AI coding assistants (ChatGPT, Copilot, DeepSeek) to generate functions, boilerplate, or refactor algorithms.

- **Licensing Goal:** Determine if using AI coding tools restricts your open-source license choices.
- **JLA Filter Focus:** Driven by human author intent (e.g.,  Commercial use +  Incl. Copyright)
- **User Obligation:** Standard obligations apply based on the license you choose for your human-authored codebase.

✓ Solution

🔧 Scenario 9: Including AI prompt templates in LLM applications

Legal Reality: Pure AI outputs lacking human oversight are generally ineligible for copyright. However, when you guide, refine, and integrate AI code into a project your repository contains Python scripts alongside complex, 500+ word structured prompt templates (system prompts, XML schemas, reasoning frameworks).

- **Licensing Goal:** As a prompt engineer, DeepSeek or OpenAI code generated using open-source licenses follows standard copyright rules (human creative oversight determines ownership).
- **JLA Filter Focus:** Treat engineered prompts as code assets:  Commercial use and  Incl. Copyright .always review the **Model Weights License** (e.g., OpenRAIL or specific commercial restrictions) attached to the LLM itself. When collaborating

✓ Solution tionally or using Asian open-source software, you may also encounter

MulanPSL-2.0 (an OSI-approved Chinese permissive license compatible with **Legal Reality:** Short functional prompts carry no copyright. However, complex, highly structured prompt templates meet the threshold of creative human expression and are legally protected as literary text assets.

- **Outcome: Fully Permissible.** Using AI tools does not force a specific open-source license onto your repository.
- **JLA Matches:** MIT , Apache-2.0 , EUPL-1.2 , GPL-3.0 (Author choice).
- **Outcome: Fully Coverable.** Engineered prompt templates checked into your repository are covered under your overall repository license.
- **JLA Matches:** MIT , Apache-2.0 , BSD-3-Clause

- **User Obligation:** Standard obligations apply based on the license you choose for your human-authored codebase.

✓ **Solution**

Scenario 9: Including AI prompt templates in LLM applications

Legal Reality: Pure AI outputs lacking human authorship are generally ineligible for copyright. However, when you guide, refine, and integrate AI code into a project your repository contains Python scripts alongside complex, 500+word structured prompt templates (system prompts, XML schemas, reasoning frameworks).

- **Licensing Goal:** Entire prompt templates are legally covered under the same open-source license as your code.
- **JLA Filter Focus:** Treat engineered prompts as code assets. **Commercial use** and **Incl. Copyright** always review the **Model Weights License** (e.g., OpenRAIL or specific commercial restrictions) attached to the LLM itself. When collaborating

tionally or using Asian open-source software, you may also encounter

MulanPSL-2.0 (an OSI-approved Chinese permissive license compatible with **Legal Reality:** Short functional prompts carry no copyright. However, complex, highly

structured prompt templates meet the threshold of creative human expression and are legally protected as literary text assets.

- **Outcome: Fully Permissible.** Using AI tools does not force a specific open-source license onto your repository.
- **JLA Matches:** **MIT**, **Apache-2.0**, **EURL-1.2**, **GPL-3.0** (Author choice).
- **Outcome: Fully Coverable.** Engineered prompt templates checked into your repository are covered under your overall repository license.
- **JLA Matches:** **MIT**, **Apache-2.0**, **BSD-3-Clause**
- **User Obligation:** Downstream users who copy your prompt files must preserve your copyright notice and file headers (**# SPDX-License-Identifier: MIT**).

Best Practice:

In-File Identification using SPDX

Once you select a license, apply it to individual source files and build recipes using **SPDX identifiers** (Software Package Data Exchange). Managed by the Linux Foundation, an SPDX identifier is a standardized, machine-readable short tag (e.g., **MIT**, **Apache-2.0**, **GPL-3.0-only**, **0BSD**) recognized by automated compliance scanners, package managers, and CI/CD build pipelines.

Instead of pasting long legal texts at the top of every file, add a single-line comment at the very first line of your script or recipe:

- In a container recipe

```
# SPDX-License-Identifier: MIT
FROM ubuntu:24.04
```

Is putting software on GitHub/GitLab/... publishing?

```
# SPDX-License-Identifier: 0BSD
import numpy as np
```



FAIR principles. (c) Scriberia for The Turing Way, CC-BY.

Is it enough to make the code public for the code to remain **findable and accessible**?

- No. Because nothing prevents me from deleting my GitHub repository or rewriting the Git history and we have no guarantee that GitHub will still be around in 10 years.

Is putting software on GitHub/GitLab/... publishing?

```
In a python script
# SPDX-License-Identifier: 0BSD
import numpy as np
```



FAIR principles. (c) [Scriberia](#) for [The Turing Way](#), CC-BY.

Is it enough to make the code public for the code to remain **findable and accessible**?

- No. Because nothing prevents me from deleting my GitHub repository or rewriting the Git history and we have no guarantee that GitHub will still be around in 10 years.
- **Make your code citable and persistent:** Get a persistent identifier (PID) such as DOI in addition to sharing the code publicly, by using services like [Zenodo](#) or similar services.

How to make your software citable

Discussion (Citation-1): Explain how you currently cite software

- Do you cite software that you use? How?
- If I wanted to cite your code/scripts, what would I need to do?

Checklist for making a release of your software citable:

- ✓ Assigned an appropriate license
- ✓ Described the software using an appropriate metadata format
- ✓ Clear version number
- ✓ Authors credited
- ✓ Procured a persistent identifier
- ✓ Added a recommended citation to the software documentation

This checklist is adapted from: N. P. Chue Hong, A. Allen, A. Gonzalez-Beltran, et al., Software Citation Checklist for Developers (Version 0.9.0). Zenodo. 2019b. ([DOI](#))

Our practical recommendations:

- [Web form to create, edit, and validate CITATION.cff files](#)
- [Video: "How to create a CITATION.cff using cffinit"](#)
- Add a file called [CITATION.cff \(example\)](#)
- Get a [digital object identifier \(DOI\)](#) for your code on [Zenodo \(example\)](#).

Where can I publish papers for how to make my GitHub project citable using Zenodo? Great list. Make it as easy as possible to do this. I don't know what you want cited.

([Neil P. Chue Hong](#))

This is an example of a simple `CITATION.cff` file:

How to cite software

```
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
authors:
  - family-names: Doe
    given-names: Jane
    orcid: https://orcid.org/1234-5678-9101-1121
title: "My Research Software"
version: 2.0.4
doi: 10.5281/zenodo.1234
date-released: 2021-08-11
```

• N. P. Chue Hong, A. Allen, A. Gonzalez-Beltran, et al., Software Citation Checklist for Authors (Version 0.9.0). Zenodo. 2019a. ([DOI](#))

More about `CITATION.cff` files.

- N. P. Chue Hong, A. Allen, A. Gonzalez-Beltran, et al., Software Citation Checklist for Developers (Version 0.9.0). Zenodo. 2019b. ([DOI](#))
- [GitHub now supports CITATION.cff files](#)

Our practical recommendations:

- [Web form to create, edit, and validate CITATION.cff files](#)
 - [Video: "How to create a CITATION.cff using cffinit"](#)
 - Add a file called [CITATION.cff](#) (example).
- Papers with focus on scientific software**
- Get a [digital object identifier \(DOI\)](#) for your code on [Zenodo](#) (example).

[Step-by-step guide](#) for how to make your GitHub project citable using [Zenodo](#)

Where can I publish papers which are primarily focused on my scientific software? Great list, [Make it as easy as possible to clearly say what you want cited](#)

[What you should I publish my software?"](#)

([Neil P. Chue Hong](#))

This is an example of a simple `CITATION.cff` file:

How to cite software

```
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
authors:
  - family-names: Doe
    given-names: Jane
    orcid: https://orcid.org/1234-5678-9101-1121
title: "My Research Software"
version: 2.0.4
doi: 10.5281/zenodo.1234
date-released: 2021-08-11
```

- N. P. Chue Hong, A. Allen, A. Gonzalez-Beltran, et al., Software Citation Checklist for Authors (Version 0.9.0), Zenodo, 2019a. ([DOI](#))
- N. P. Chue Hong, A. Allen, A. Gonzalez-Beltran, et al., Software Citation Checklist for Developers (Version 0.9.0), Zenodo, 2019b. ([DOI](#))

More about `CITATION.cff` files.

- [GitHub now supports CITATION.cff files](#)

Recommended format for software citation is to ensure the following information is provided as part of the reference (from [Katz, Chue Hong, Clark, 2021](#) which also contains software citation examples):

- Creator
- Title
- Publication venue
- Date
- Identifier
- Version
- Type

 **FAIR for Research Software (FAIR4RS)**

The FAIR4RS Principles adapt the original FAIR data principles to improve the **Findability, Accessibility, Interoperability, and Reusability** of research software. They provide guidelines to make research software more open, discoverable, citable, and reusable, which supports reproducibility and open science. The summary of the FAIR4RS below is a compressed version of the principles listed in [Barker, Michelle, et al. "Introducing the FAIR Principles for research software." Scientific Data 9.1 \(2022\): 622](#).

- **Findable:** Software should have a globally unique and persistent identifier (e.g. a DOI from Zenodo when depositing a software release, software metadata in citation file).

Sharing data

Services for sharing and collaborating on research data

Interoperable software should use data formats and standards [Brought with Research Data Repositories \(re3data\)](#) platform and filter by country, content type, discipline, etc. (e.g. input/output files in CSV, dependencies listed in `requirements.txt` or `environment.yml`, configuration files in yaml).

International:

- **Reusable:** Software should have clear licensing, documentation, and provenance (e.g. a standard license and a `README` with usage instructions, authors listed with ORCIDs).
- **Zenodo**: A general-purpose open access repository created by OpenAIRE and CERN. Integration with GitHub, allows researchers to upload files up to 50 GB.
- **Figshare**: Online digital repository where researchers can preserve and share their research outputs (figures, datasets, images and videos). Users can make all of their research outputs available in a citable, shareable and discoverable manner.

There are great resources to self-evaluate the FAIRness of your research software:

- **FAIR Software Checklist** which provides a questionnaire and even a badge of how FAIR the software is
- **EURO-AT**: European platform for researchers and practitioners from any research discipline to preserve, find, access, and process data in a trusted environment.
- **FAIR Software NL** highlights with nice visuals the five most important elements of FAIR4RS (1. Public accessible repository with version control; 2. License; 3. Software registry; 4. Software citation file; 5. Software quality checklist)

- **The Open Science Framework**: Gives free accounts for collaboration around files and other research artifacts. Each account can have up to 5 GB of files without any problem,

Sharing data

including reasoning of releases, DOI from Zenodo when depositing a software release, software metadata in citation file).

Services for sharing and collaborating on research data

services: software and metadata sharing and collaboration protocols (e.g. downloading from GitHub or Zenodo).

To find **interoperable** software, you should use data format and standards **to work with Research tools**

Data Repositories (re3data) platform and filter by country, content type, discipline, etc. (e.g. input/output files in CSV, dependencies listed in `requirements.txt` or

`environment.yml`, configuration files in yaml).

- **Reusable**: Software should have clear licensing, documentation, and provenance (e.g. a standard license and a `README` with usage instructions, authors listed with ORCID).
- **Zenodo**: A general purpose open access repository created by OpenAIRE and CERN. Integration with GitHub, allows researchers to upload files up to 50 GB.
- **Figshare**: Online digital repository where researchers can preserve and share their research outputs (figures, datasets, images and videos). Users can make all of their research outputs available in a citable, shareable and discoverable manner.
- **FAIR Software Checklist**: which provides a questionnaire and even a badge of how FAIR the software is
- **EUDAT**: European platform for researchers and practitioners from any research discipline to preserve, find, access, and process data in a trusted environment.
- **FAIR Software NL** highlights with nice visuals the five most important elements of FAIR: 1. Public accessible repository with version control; 2. License; 3. Software non profit membership or organization. A curated resource that makes the data underlying scientific publications discoverable, freely reusable, and citable.
- **The Open Science Framework**: Gives free accounts for collaboration around files and other research artifacts. Each account can have up to 5 GB of files without any problem, and it remains private until you make it public.

Sweden:

- [Researchdata.se](#) - research data from a wide range of disciplines
- [ICOS for climate data](#)
- [Bolin center climate / geodata](#)
- [NBIS for life science, sequence, omics data](#)

Norway:

- [NIRD archive is widely used by some communities](#)
- [NSD - Norwegian Center for Research Data](#), for any kind of data
- [Dataverse.no](#) - Dataverse network, based at University of Tromsø but open for other institutions
- [ELIXIR Norway for life science, sequence, omics data](#)

Denmark:

- [Datavejviser](#) – metadata catalogue aggregating from research organisations and National Archives

[CEUS Data Explorer](#) - Data from the Geological Survey of Denmark and Greenland

- [DTU Data](#) – repository of the Technical University of Denmark
- [The EU has a database directive](#) which restricts data mining on databases.
- Has a somewhat similar effect to copyright, because copyright would not apply to data mining.

Finland:

- A good license also gives rights to data mine. So not a major concern.

- [IDA, for general data](#)

When you can use datasets:

- [FSD for social data](#)
- [more information on Finnish resources for open data](#)
- The license allows
- Your country has exceptions for research

Portugal:

- The data doesn't come from the EU

- [DMPortal - Dataverse network for any research data](#)

License text, slides, images, and supporting information under a [Creative Commons license](#), and get a DOI using [Zenodo](#) or [Figshare](#) or [OSF](#) other services.

Resources for data management

Licensing and machine learning/ AI

- [DataLad](#)

- [Data Stewardship Wizard](#)

This section is maybe more relevant to **developers of AI models / AI systems** rather than **users of AI models / AI systems**.

Licensing of datasets and databases

• [CEUS Data Portal](#) – data from the Geographical Survey of Denmark and Greenland

- [DTU Data](#) – repository of the Technical University of Denmark
- [The EU has a data directive](#) which restricts data mining on databases.
- Has a somewhat similar effect to copyright, because copyright would not apply to data mining.

Finland:

- A good license also gives rights to data mine. So not a major concern.

- [IDA, for general data](#)

When you can use datasets:

- [FSD for social data](#)
- [more information on Finnish resources for open data](#)
- The license allows
- Your country has exceptions for research

Portugal:

- The data doesn't come from the EU

• [DMPortal - Dataverse network for any research data](#)
License text, slides, images, and supporting information under a [Creative Commons license](#), and get a DOI using [Zenodo](#) or [Figshare](#) or [OSF](#) other services.

Resources for data management

Licensing and machine learning/ AI

- [DataLad](#)
- [Data Stewardship Wizard](#)

This section is maybe more relevant to **developers of AI models / AI systems** rather than **users of AI models / AI systems**.

Is it data? Is it software? It depends. We need to consider the AI system as a whole, the training data, the production data, the AI output, and how it is put on service. **AI models** are like the engine of the car: they cannot do anything without the rest of the car infrastructure. **AI systems** are the whole car with the AI model and all the software and hardware to actually use it.

Depending on what you are going to share, there might be things to consider beyond the license.

For example **large language models** are often shared with open source software licenses, on **HuggingFace** which is like a GitHub/GitLab for AI models (see for example the [OLMO model](#)). Many so-called *open-source* models are actually just *open-weights* models: only the trained neural network weights are shared, while the training data, training code, and full documentation are often kept private. This lack of transparency raises concerns about reproducibility and accountability and this phenomenon is sometimes called “**open washing**” ([ref](#)). Models are also shared with a **model card** which is a documentation tool for transparency that provide a comprehensive snapshot of a model's characteristics and ethical considerations (see [Ch.8 Glerean 2025](#)).

What about ethics? What about liability?

As AI models (e.g. the deep network weights) and AI systems (the model with all the software and hardware to query it) are becoming more available, there can be legal (and ethical!) requirements on the developer of the AI model/system by the [EU AI Act](#). In general

Further reading

- [The Turing way](#)
- [Illustrations from the Turing Way book dashes](#)
- [Reproducibility syllabus](#)
- [The reproducible research data analysis platform](#)

What about the training data inside the model? Large models can memorize and unintentionally reveal parts of their training data. This raises concerns about copyright, trade secrets, and personal data. News publishers and artists are suing AI companies for unauthorized use of their content in training. It is still unclear how traditional data licenses can apply to data that has been transformed into model weights.

- [top 10 FAIR Data & Software Things](#) are brier guides that can be used by the research community to understand how they can make their research (data and software) more FAIR.

More resources

- [Five recommendations for fair software](#)
- [RAIL initiative: “Responsible AI licenses”](#)
- [Publishing research software](#) – A MIT libraries webpage on why to publish software,
- [The Turing Way: Machine Learning Model Licenses](#) where to publish software, and how to make software citable.
- [“Expert Q&A on Artificial Intelligence \(AI\) Licensing”](#)
- [Software Quality Checklist](#)
- [MoSSI Best Practice Guides](#)
- [Five recommendations for fair software](#)
- [Awesome Research Software Registries](#)

Who is the course for?

Further reading

and infrastructure to query it) are becoming more available, there can be legal (and ethical!) requirements on the developer of the AI model/system by the [EU AI Act](#)[🔗]. In general researchers do not need to worry, but ethically one should consider that if the research-purpose AI model/system could be used for something harmful, ethically (if not legally) one should consider if such model/system should be implemented at all.

- [Reproducibility syllabus](#)[🔗]

The [reproducible research data analysis platform](#)[🔗]

What about the training data inside the model? Large models can memorize and unintentionally reveal parts of their training data. This raises concerns about copyright, trade secrets, and personal data. News publishers and artists are suing AI companies for

unauthorized use of their content in training. It is still unclear how traditional data licenses can apply to data that has been transformed into model weights.

- [Top 10 FAIR Data & Software Things](#)[🔗] are brier guides that can be used by the research community to understand how they can make their research (data and software) more FAIR.

More resources

- [Five recommendations for fair software](#)[🔗]
- [RAIL initiative: “Responsible AI licenses”](#)[🔗]
- [Publishing research software](#)[🔗] A MIT libraries webpage on why to publish software,
- [The Turing Way: Machine Learning Model Licenses](#)[🔗] where to publish software, and how to make software citable.
- [“Expert Q&A on Artificial Intelligence \(AI\) Licensing”](#)[🔗]
- [Software Quality Checklist](#)[🔗]

- [MolSSI Best Practice Guides](#)[🔗]

- [Five recommendations for fair software](#)[🔗]

- [Awesome Research Software Registries](#)[🔗]

Who is the course for?

- Someone writing their own relatively small material
- Someone who wants to incorporate other code/libraries into their own projects
- Group leader who wants to decide how to balance openness and long-term strategy

List of exercises

Full list

This is a list of all exercises and solutions in this lesson, mainly as a reference for helpers and instructors. This list is automatically generated from all of the other pages in the lesson. Any single teaching event will probably cover only a subset of these, depending on their interests.

Instructor guide

Basics

This is basically presented as-is - not much technical presentation is needed.

There are the following main acts:

- First section is about social coding and sharing code and motivation for sharing code with a discussion that licensing alone isn't enough. If you want people to use your work, you need to make sure to make your work easily reusable.

This discussion is called “software licensing”. You could mention that to make it more interesting, we have adopted the current boiler name, and our emphasis is not just licensing, but why licensing is needed.

Emphasis of the concept of derivative work and that everything is based on other things.

And if your goal is citations, you want as many people using your work as possible, because then they can cite you.

Software licensing

- **Objectives** Overview of license types. There are many online resources, and the biggest question is

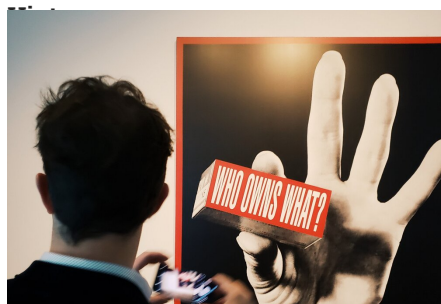
permissive vs. weak copyleft vs. strong copyleft. The [cake analogy for licenses](#)[🔗] is only linked here can be skipped, derivative work is not in and whether we can share it.

- **Get familiar with terminology around licensing** Get familiar with terminology around licensing

- **Practical advice for software licensing** Practical advice for software licensing.

- The “Sharing data” part is meant to bring awareness about FAIRness and Open Science and to show how DOIs can be obtained for Git repositories.

Copyright



the particular good or bad cases you know

to use something because licensing wasn't

plans?

a discussion that licensing alone isn't enough. If you want people to use your work, you need to do more to make your work easily reusable. Emphasis of the concept of derivative work and that everything is based on other things. And if your goal is citations, you want as many people using your work as possible, because then they can cite you.

Software licensing

- Overview of license types. There are many online resources, and the biggest question is permissive vs. weak copyleft vs. strong copyleft. The cake analogy for licenses is only linked and can be skipped and does not have to be discussed in detail.
- Get familiar with terminology around licensing.
- Then a brief section on code citations. It's good if you manage to start a discussion around this topic.
- The "Sharing data" part is meant to bring awareness about FAIRness and Open Science and to show how DOIs can be obtained for Git repositories.

Copyright



the particular good or bad cases you know

to use something because licensing wasn't

plans?

- **Trademark:** Protects a name/brand from impersonation.
- **Patent:** Protects a novel, non-obvious, technical invention.
- **Copyright:** Protects **creative expression**: software, writing, graphics, photos, certain datasets, this presentation. Practically "forever" (lifetime of author + 70 years).

Copyright controls whether and how we can distribute the original work or the **derivative work**.

Derivative work: Sampling/remixing



[Midjourney, CC-BY-NC 4.0]

- Changing and distributing software is similar to changing and distributing music
- You can do almost anything if you don't distribute it

Often we don't have the choice:

[Midjourney, CC-BY-NC 4.0]

- We are expected to publish software
- Sharing can be an insurance against being locked out

Exercise: Derivative work

💬 Licensing-1: What constitutes derivative work?

This question 5 below can be used as a starting point and copied to the collaborative document to form input for an online poll:

```
## Question 5: Which of these are derivative works?

**Choose many**. Vote by adding an `o` character:

- A. Download some code from a website and add on to it
  - votes:

- B. Download some code and use one of the functions in your code
```

[Midjourney, CC-BY-NC 4.0]

- Changing and distributing software is similar to changing and distributing music
- You can do almost anything if you don't distribute it

Often we don't have the choice:

[Midjourney, CC-BY-NC 4.0]

- We are expected to publish software
- Sharing can be as if insurance against being locked out

Exercise: Derivative work

💬 Licensing-1: What constitutes derivative work?

This question 5 below can be used as a starting point and copied to the collaborative document form input for an online poll:

Question 5: Which of these are derivative works?

****Choose many****. Vote by adding an `o` character:

- A. Download some code from a website and add on to it
- votes:
- B. Download some code and use one of the functions in your code
- votes:
- C. Changing code you got from somewhere
- votes:
- D. Extending code you got from somewhere
- votes:
- E. Completely rewriting code you got from somewhere
- votes:
- F. Rewriting code to a different programming language
- votes:
- G. Linking to libraries (static or dynamic), plug-ins, and drivers
- votes:
- H. Clean room design (somebody explains you the code but you have never seen it)
- votes:
- I. You read a paper, understand algorithm, write own code
- votes:

✓ Solution

- Derivative work: A-F
- Not derivative work: G-I
- E and F: This depends on how you do it, see clean room design.

copyright anymore). Copyright infringement instead is illegal and it can result in criminal charges (e.g. fines). Copyright however protects the particular expression of an idea or fact (for example, the specific source code of a program, but not the underlying algorithm itself).

This insert can be skipped and left as reading exercise

There is no pre-defined “number of lines of code”, “seconds of a song”, or “pixels of an image” that can be used as the basis for considering a particular work as derivative in the context of research, it can be possible to use Quotation Exception (in EU, [ref](#)) and Fair use (in USA, [ref](#)). Plagiarism is the practice of taking somebody else’s ideas or work and claim them as your own: it is the **unacknowledged** use of another person’s work. Intellectual Property Rights (IPRs) infringement instead is the **unauthorised** use of another’s work. Henderson, P., Li, X., Jurafsky, D., Hashimoto, T., Lemley, M. A., & Liang, P. (2023).

Foundation models and fair use. *Journal of Machine Learning Research*, 24(400), 1-79. [↗](#)) IPRs can be classified in two main groups (WTO [↗](#)): i) Copyright and rights related to

Derivative work and containers

Copyright (computer programs are here) and ii) Industrial properties like trademarks, and inventions (which may include specific technical implementations of systems or code) protected by patents.

Containers are a bit more tricky when it comes to licenses.

1. Distribution of container recipes: it's like distributing source code.

In research ethics, plagiarism is one of the three definition of research misconduct (along with fabrication and falsification, see [ACPEA, European Code of Conduct for Research Integrity](#)). Plagiarism is not illegal per se, but it can lead to serious consequences like the retraction of published work. One can engage in plagiarism, without necessarily breaking any IPR law (e.g. write a new book by reusing the plot of an old book that is not under “Creative Commons” or [CC-BY-NC](#)). But if the contributor insists first-hand on retracts

copyright anymore). Copyright infringement instead is illegal and it can result in criminal charges (e.g. fines). Copyright however protects the particular expression of an idea or fact (for example, the specific source code of a program, but not the underlying algorithm itself).

This insert can be skipped and left as reading exercise

There is no pre-defined “number of lines of code”, “seconds of a song”, or “pixels of an image” and it is important to consider also **plagiarism** and how it relates to the academic context. It is possible to use **Quotation Exception** (in EU, [ref](#)) and **Fair use** (in USA, [ref](#)). Plagiarism is the practice of taking somebody else’s ideas or work and claim them as your own; it is the **unacknowledged** use of another person’s work. Intellectual Property Rights (IPRs) infringement instead is the **unauthorised** use of another’s work. [Henderson, T., Li, X., Jurafsky, D., Hashimoto, T., Lemley, M. A., & Liang, P. \(2023\).](#)

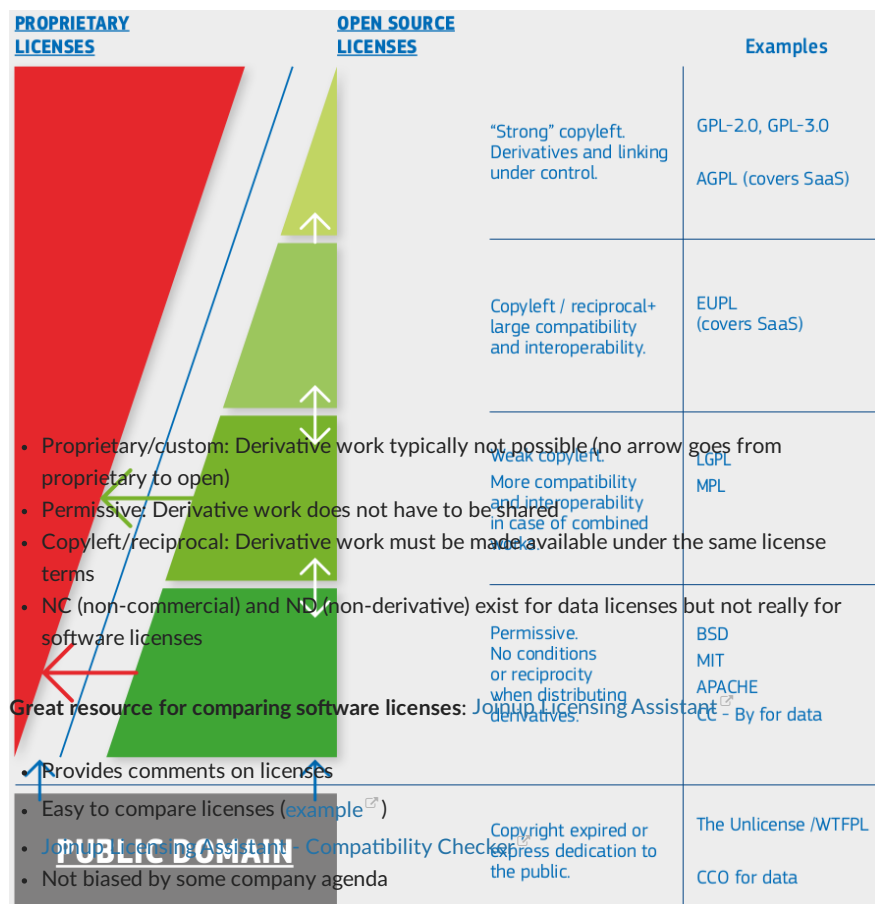
Foundation models and fair use. *Journal of Machine Learning Research*, 24(400), 1-79. [\(ref\)](#) IPRs can be classified in two main groups ([WTO](#) [\(ref\)](#)): i) Copyright and rights related to

Derivative work and containers

Copyright (computer programs and here) and ii) Industrial properties like trademarks, and inventions (which may include specific technical implementations of systems or code) Containers are a bit more tricky when it comes to licenses.

In research ethics, plagiarism is one of the three definition of research misconduct (along with fabrication and falsification, see [ACAA](#), [European Code of Conduct for Research Integrity](#)). Plagiarism is not illegal per se, but it can lead to serious consequences like the retraction of published work. One can engage in plagiarism, without necessarily breaking any IPR law (e.g. write a new book by reusing the plot of an old book that is not under “Mere Aggregation” at [GPL-FAQ](#) [\(ref\)](#)). Briefly, if the container image just bundles separate programs that talk through normal system interfaces, it is an **aggregate** and each keeps its own license (like a CD-ROM with various packages). If the components are tightly integrated into one program (e.g. a pipeline with various parts that the container can run as a single program), the image may be treated as a **derivative work**, and stricter license obligations (e.g. GPL copyleft) can apply.

Taxonomy of software licenses



European Commission, Directorate-General for Informatics, Schmitz, P., *European Union Public Licence (EURL): guidelines July 2021*, Publications Office, 2021, and open source explained with cakes [\(ref\)](#)
<https://data.europa.eu/doi/10.2799/77160> [\(ref\)](#)

Exercise: Licensing situations

Comments:

Licensing-2: Consider some common licensing situations

- Arrows represent compatibility (A → B: B can reuse A)
- 1. What is the Stack Overflow license for code you copy and paste?

- Proprietary/custom: Derivative work typically not possible (no arrow goes from proprietary to open) - Permissive: Derivative work does not have to be shared - Copyleft/reciprocal: Derivative work must be made available under the same license terms - NC (non-commercial) and ND (non-derivative) exist for data licenses but not really for software licenses	Weak copyleft More compatibility and interoperability in case of combined use Permissive. No conditions or reciprocity when distributing derivatives.	GPL MPL BSD MIT APACHE CC - By for data
Great resource for comparing software licenses: Joinup Licensing Assistant - Provides comments on licenses - Easy to compare licenses (example) Joinup Licensing Assistant - Compatibility Checker - Not biased by some company agenda	Copyright expired or express dedication to the public.	The Unlicense /WTFPL CCO for data

European Commission, Directorate-General for Informatics, Schmitz, P., European Union Public Licence (EURL): guidelines July 2021, Publications Office, 2021, <https://data.europa.eu/doi/10.2799/77160>

Exercise: Licensing situations

Comments:

Licensing-2: Consider some common licensing situations

- Arrows represent compatibility (A & B can reuse A)
- 1. What is the StackOverflow license for code you copy and paste?
- 2. A journal requests that you release your software during publication. You have copied a portion of the code from another package, which you have forgotten. Can you satisfy the journal's request?
- 3. You want to fix a bug in a project someone else has released, but there is no license. What risks are there?
- 4. How would you ask someone to add a license?
- 5. You incorporate MIT, GPL, and BSD3 licensed code into your project. What possible licenses can you pick for your project?
- 6. You do the same as above but add in another license that looks strong copyleft. What possible licenses can you use now?
- 7. Do licenses apply if you don't distribute your code? Why or why not?
- 8. Which licenses are most/least attractive for companies with proprietary software?

✓ Solution

- As indicated [here](#), all publicly accessible user contributions are licensed under [Creative Commons Attribution-ShareAlike](#) license. See Stackoverflow [Terms of service](#) for more detailed information.
- "Standard" licensing rules apply. So in this case, you would need to remove the portion of code you have copied from another package before being able to release your software.
- By default you are not authorized to use the content of a repository when there is no license. And derivative work is also not possible by default. Other risks: it may not be clear whether you can use and distribute (publish) the bugfixed code. For the repo owners it may not be clear whether they can use and distributed the bugfixed code. However, the authors may have forgotten to add a license so we suggest you to contact the authors (e.g. make an issue) and ask whether they are willing to add a license.
- As mentioned in 3., the easiest is to fill an issue and explain the reasons why you would like to use this software (or update it).
- Combining software with different licenses can be tricky and it is important to understand compatibilities (or lack of compatibilities) of the various licenses. GPL license is the most protective (BSD and MIT are quite permissive) so for the resulting combined software you could use a GPL license. However, re-licensing may not be necessary.
- Derivative Work would need to be shared under this strong copyleft license (e.g. AGPL or GPL), unless the components are only plugins or libraries.
- If you keep your code for yourself, you may think you do not need a license.

When should I add a license?

- Choose a license early in the project, even before you publish it. Later in the project it may become complicated to change it. Agreeing on a software license does not mean that you have to make it open immediately. You can also follow the "open core" approach: You don't have to open source all your work. Core can be open and on a public branch. Unpublished code can be on a private repository.
- Combining software with different licenses can be tricky and it is important to understand compatibilities (or lack of compatibilities) of the various licenses. GPL license is the most protective (BSD and MIT are quite permissive) so for the resulting combined software you could use a GPL license. However, re-licensing may not be necessary.

However, we recommend to work as if the code is public even though it still may be private (thanks to F. Glerum for this great suggestion): This is to avoid surprises about code in the history with incompatible license years later when you decide to open the project.

However, remember that in most companies/universities, your employer is

How to add a license if your work is derivative work

- owning your work and when you leave you may not be allowed to "distribute your code to your future self". So the best is always to add a license!
- Your code is derivative work if you have started from an existing code and made changes to it
8. The least attractive licenses for companies with proprietary software are licenses or if you incorporated an existing code into your code where you would need to keep an open license when creating derivative work. For instance GPL and AGPL. The most attractive licenses are permissive licenses. If your code is derivative work, then you need to check the license of the original code. Depending on the license, your choices might be limited. In this case we recommend to use

where they can reuse, modify and relicense with no conditions. For instance MIT, BSD and Apache license.

the repo owners it may not be clear whether they can use and distributed the bugfixed code. However, the authors may have forgotten to add a license so we

suggest you to contact the authors (e.g. make an issue) and ask whether they are willing to add a license.

When should I add a license?

4. As mentioned in 3., the easiest is to fill an issue and explain the reasons why you **Choose a license early in the project, even before you publish it.** Later in the project it may become complicated to change it. Agreeing on a software license does not mean that you

5. Combining software with different licenses can be tricky and it is important to have to make it open immediately. You can also follow the **"open core" approach:** You don't understand compatibilities (or lack of compatibilities) of the various licenses. GPL license is the most protective (BSD and MIT are quite permissive) so for the code can be on a private repository. Unpublished code can be on a private repository. Resulting combined software you could use a GPL license. However, re-licensing may not be necessary.

However, we recommend to **work as if the code is public even though it still may be private** 6. Derivative work would need to be shared under this strong copyright license (e.g. AGPL or GPL), unless the components are only plugins or libraries.

(thanks to S. Clercan for this great suggestion). This is to avoid surprises about code in the history with incompatible license years later when you decide to open the project.

7. If you keep your code for yourself, you may think you do not need a license.

However, remember that in most companies/universities, your employer is

How to add a license if your work is derivative work

owning your work and when you leave you may not be allowed to "distribute

your code to your future self". So the best is always to add a license!

Your code is derivative work if you have started from an existing code and made changes to it

8. The least attractive licenses for companies with proprietary software are licenses where you would need to keep an open license when creating derivative work. For

instance GPL and and AGPL. The most attractive licenses are permissive licenses

If your code is derivative work, then **you need to check the license of the original code.**

Depending on the license, your choices might be limited. In this case we recommend to use these two resources:

- [Joinup Licensing Assistant - Find and compare software licenses](#)
- [Joinup Licensing Assistant - Compatibility Checker](#)

If the original code does not have a license, you may not be able to distribute your derivative code. You can try to contact the authors and ask them to clarify the license of their code.

Practical steps for **incorporating something small into your own project** with a license that allows you to do so (as an example incorporating a function or two from another project):

- Create a `LICENSES/` folder in your project and "put the unmodified license text (i.e., the license text template without any copyright notices) in your `LICENSES/` folder" (<https://reuse.software/faq/#license-templates>). This way if you reuse code from multiple projects, you can keep there multiple license files.
- **Put the code that you incorporate into a separate file or separate files.** This makes it later easier to see what was incorporated, and what was written from scratch. On top of the file(s) which you have incorporated into your project add (and adapt) the following header ([more examples](#)):

```
# SPDX-FileCopyrightText: 2023 Jane Doe <jane@example.com>
#
# SPDX-License-Identifier: MIT
```

- helpful for others if you do. You can state your changes in the header of the files you have modified. It can be helpful to state bigger-picture changes in the README file of the project. The [REUSE](#) initiative was started by the [Free Software Foundation Europe](#) to make licensing of software projects easier, but is OK if you prefer to not follow this strict format but the advantage of the changes (share-alike) [reuse-tool](#) makes it then easy to verify and update license headers if you have many files from different sources.

If your work is not derivative work

- If it does not make sense to have several files in your project (e.g. when incorporating something into a notebook), then add a note/comment about the license and where the code came from on top of the function. If you have started "from scratch", and not used any existing code, or incorporated existing code into your code, then you may consider your code to be not derivative work.

- Although it is not dictated by the license but it can still be nice to acknowledge the incorporated functions/code in your README/documentation and to cite their work if Before you may choose a license, clarify the following points with, for example, your supervisor, collaborators, or principal investigator:

- Some licenses are more permissive (you can keep your changes private) but some licenses require you to publish the changes (share-alike).
 - Does your work contract, grant, or collaboration agreement dictate a specific license?
 - Is there an intent to commercialize the code?
- Practical steps for making **changes to an existing project** with a license that allows you to do so:
- When there is unknown or mixed ownership: If there are multiple persons or organizations as owners of the code, all must agree to the license.

• If the project is on GitHub or GitLab or similar, first fork the project (copy it into your user space where you can make changes). **Do not invent your own license.** Choose one of the standard licenses, otherwise compatibility is not clear.

- `# SPDX-License-Identifier: MIT`

helpful for others if you do. You can state your changes in the header of the files you have modified. It can be helpful to state bigger-picture changes in the README file of the project. The [REUSE](#) initiative was started by the [Free Software Foundation Europe](#) to make licensing of software projects easier. It is OK if you prefer to not follow this strict format but the advantage of following it is that the [reuse-tool](#) makes it then easy to verify and update license headers if you have many files from different sources.

If your work is not derivative work

- If it does not make sense to have several files in your project (e.g. when incorporating something into a notebook), then add a note/comment about the license and where the code came from on top of the function.
- If you have started "from scratch", and not used any existing code, or incorporated existing code into your code, then you may consider your code to be not derivative work.
- Although it is not dictated by the license but it can still be nice to acknowledge the incorporated functions/code in your README/documentation and to cite their work if you publish a paper about your code.
- Before you may choose a license, clarify the following points with, for example, your supervisor, collaborators, or principal investigator:
 - Some licenses are more permissive (you can keep your changes private) but some licenses require you to publish the changes (share-alike).
 - Does your work contract, grant, or collaboration agreement dictate a specific license?
 - Is there an intent to commercialize the code?
- Practical steps for making changes to an existing project with a license that allows you to do so:
 - When there is unknown or mixed ownership: If there are multiple persons or organizations as owners of the code, all must agree to the license.

If the project is on GitHub or GitLab or similar, first fork the project (copy it into your user space where you can make changes). Choose one of the standard licenses, otherwise compatibility is not clear.

- [Joinup Licensing Assistant - Find and compare software licenses](#)
- [Joinup Licensing Assistant - Compatibility Checker](#)

Practical steps:

- Create a `LICENSES/` folder (example).
- Put the unmodified license text (i.e., the license text template without any copyright notices) in plain text format into the folder (example). Here are the two above licenses in plain text: [EUPL](#) and [MIT](#) (but the latter contains a copyright notice which we rather want to have on top of files).
- Add copyright and license information to each file following <https://reuse.software/tutorial/> which uses a standard format with so-called [SPDX identifiers](#). Example below (example):

```
# SPDX-FileCopyrightText: 2023 Jane Doe <jane@example.com>
#
# SPDX-License-Identifier: EUPL-1.2
```

The [REUSE](#) initiative was started by the [Free Software Foundation Europe](#) to make licensing of software projects easier. It is OK if you prefer to not follow this strict format

but the advantage of following it is that the [reuse-tool](#) makes it then easy to verify and update license headers if you have many files from different sources.

- Risks related to licenses/IPR: you have generated code that is actually verbatim copy of fully copyrighted code, or code that requires a strict copyleft license. Plagiarism
- For really small projects with one or two files the above may seem excessive and some projects choose to not have copyright information on top of their files and they only have one LICENSE file and that is OK for really small projects.

If we focus on the last one, a recent paper (ref.) estimates that around 2% of AI generated code is "strikingly similar to existing open-source implementations". Generative AI tools are typically not able to provide an exact reference of where certain bits of generated code were copied from, so it is the responsibility of the researcher to verify with generative AI tools for coding such as GitHub copilot, Cursor, or even basic chat that the produced code is citing and referencing the license of other published pieces of implementations (ChatGPT, Claude, Grok...). the responsibility fully lays on the person who is going to use (and publish) the generated code. You can never blame the autopilot or the company who invented it, only the driver (you!).

There are various risks in using generative AI code (this is not a taxonomy). A few

Great resources

- Risks for the derivative work: you think your code is doing what you asked, but you did not review it and your results are false
- Risks for the system in use: your generated code has software security issues, e.g. an import is a typosquat of an actual library (e.g. "microsoft" is spelled "rnicrosoft" and depending on the font you might totally miss it...)
- [Draft: Research software licensing guide](#)
- [Joinup Licensing Assistant - Find and compare software licenses](#)
- [Joinup Licensing Assistant - Compatibility Checker](#)

- But the advantage of following it is that the `reuse-tool` makes it then easy to verify and update license headers if you have many files from different sources.
 - Risks related to licenses/IPR: you have generated code that is actually verbatim copy of fully copyrighted code, or code that requires a strict copyleft license. Plagiarism
 - **Effectively also applies** to projects with one or two files the above may seem excessive and some projects choose to not have copyright information on top of their files and they only have one `LICENSE` file and that is OK for really small projects
- If we focus on the last one, a recent paper (2024) estimates that around 2% of AI generated code is “strikingly similar to existing open-source implementations”. Generative AI tools are typically not able to provide an exact reference of where certain bits of generated code were copied from, so it is the responsibility of the researcher to verify that the produced code is citing and referencing the license of other published pieces of software. With generative AI tools for coding such as GitHub copilot, Cursor, or even basic chat implementations (ChatGPT, Claude, Grok, ...) the responsibility fully lays on the person who is going to use (and publish) the generated code. You can never blame the autopilot share the same set of licenses (e.g. based only on MIT) to mitigate these risks. or the company who invented it, only the driver (you!).

There are various risks in using generative AI code (this is not a taxonomy). A few

Great resources

- **Risks for the derivative work**: you think your code is doing what you asked, but you did not review it and your results are false [Software Alliance](#)
- Risks for the system in use: your generated code has software security issues, e.g. an import is a *typosquat* of an actual library (e.g. “microsoft” is spelled “rnicrosoft” and depending on the font you might totally miss it...)
- [Draft: Research software licensing guide](#)
- [Joinup Licensing Assistant - Find and compare software licenses](#)
- [Joinup Licensing Assistant - Compatibility Checker](#)
- [Social coding lesson material](#) by [CodeRefinery](#)
- [Citation File Format \(CFF\)](#)
- [License Selector](#)
- [GitHub licensing guide](#)
- [Choosing an open-source licence](#)
- [Understanding Open Source and Free Software Licensing](#)
- [Software Licenses in Plain English](#)
- [Don’s Bibliography of Ethical Source Reading and Resources](#)
- [Mikko Välimäki: The Rise of Open Source Licensing](#)
- [Lawrence Rosen: Open Source Licensing](#)
- [Aalto IPR Cheatsheet](#)
- [Contributor License Agreements](#)
- [4OSS recommendations](#)
- [4OSS lesson](#)
- [Intellectual Property Rights \(IPR\), Licensing And Patents](#)
- [Dispelling Open Source Confusion: An Introduction to Licenses](#)
- <https://choosealicense.com> (can send automatic pull request to your GitHub repo)
- <https://hintjens.gitbooks.io/social-architecture/content/chapter2.html>
- <http://rkd.zgib.net/scicomp/open-science/open-science.html>
- <https://doi.org/10.5281/zenodo.16410766> (Stripe Press)

[Open Source Guides](#)
The machine-readable citation record is kept in `CITATION.cff` at the root of this lesson’s repository, which lists the full set of contributing authors and stays up to date as the lesson evolves.
See Christopher M. Kelly: [“Two Bits: The Cultural Significance of Free Software”](#) (Duke University Press)

Structured Metadata

- [Open Source \(Almost\) Everything](#)
- This lesson also publishes its metadata as machine-readable [Bioschemas](#) / [schema.org](#) structured data, embedded as JSON-LD on every page of this site.

All of this lesson’s metadata is generated from a single source, `metadata.yml` rendered here directly at build time:

• You cannot ignore licensing: default is “no one can make copies or derivative works”.	
Reusing	Social coding and open software - What can you do to get credit for your code and allow reuse?
This lesson material to do so.	CodeRefinery, Beckjan B. and Kerttu Thors-Wall, John van der Vliet, Richard Darst, Sabry Razick, Jyry Suvielento, Anne Fouilloux, Oxana Smirnova, Lucy Whalley, Enrico Glerean
Version	2024-08-10
How to cite this lesson	
DOI	10.5281/zenodo.16410766
If you use this lesson material, please cite it using this DOI:	
License	CC-BY-4.0

- [Nadia Asparouhova \(formerly Nadia Eghbal\): "Working in Public: The Making and Maintenance of Open Source Software"](https://doi.org/10.5281/zenodo.16410766) (Stripe Press)

The machine-readable citation record is kept in `CITATION.cff` at the root of this lesson's repository, which lists the full set of contributing authors and stays up to date as the lesson evolves. Christopher M. Kelty: "[Two Bits: The Cultural Significance of Free Software](#)" (Duke University Press)

- [Open Source \(Almost\) Everything](#)
- This lesson also publishes its metadata as machine-readable [Bioschemas](#) / [schema.org](#) structured data, embedded as JSON-LD on every page of this site.

All of this lesson's metadata is generated from a single source, `metadata.yml`, rendered here directly at build time:

- **You cannot ignore licensing:** default is “no one can make copies or derivative works”.

Reusing	Social coding and open software - What can you do to get credit for your code and allow reuse?
Authors	This lesson material is derived from CodeRefinery Packagist and Kitten . This page collects everything you need to do so. Social coding and open software by Richard Darst, Sabry Razick, Jyry Suvilehto, Anne Fouilloux, Oxana Smirnova, Lucy Whalley, Enrico Glierean
Version	2026-08-10
DOI	10.5281/zenodo.16410766
If you use this lesson material, please cite it using this DOI:	
License	CC-BY-4.0
Lesson website	https://coderefinery.github.io/social-coding
Source repository	https://github.com/coderefinery/social-coding
Keywords	software licensing, software citation
Educational level	Beginner
Language	en-UK
Is part of	https://coderefinery.org
Learning resource type	lesson

License

Website template

The website template is maintained by [CodeRefinery](#) and rendered with [sphinx-lesson: structured lessons with Sphinx](#).

Instructional material

- You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation. All CodeRefinery instructional material is made available under the [Creative Commons Attribution license \(CC-BY-4.0\)](#). The following is a human-readable summary of (and not a substitute for) the full text of the [CC-BY 4.0 license](#). No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as publicity, privacy, or moral rights may limit how you use the material.
- You are free:

- to **Share** - copy and redistribute the material in any medium or format
- to **Adapt** - remix, transform, and build upon the material

for any purpose, even commercially. The licensor cannot revoke these freedoms as long as you follow these license terms:

- **Attribution** - You must give appropriate credit (mentioning that your work is derived from work that is Copyright (c) CodeRefinery and, where practical, linking to <https://coderefinery.org>, provide a [link to the license](#), and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

No additional restrictions - You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits. With the understanding that:

- You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation.
- Attribution license (CC-BY-4.0) [↗] The following is a human-readable summary of (and not a substitute for) the full legal text of the CC-BY 4.0 license [↗]
- No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as publicity, privacy, or moral rights may limit how you use the material.

You are free:

- to **Share** - copy and redistribute the material in any medium or format
- to **Adapt** - remix, transform, and build upon the material

for any purpose, even commercially. The licensor cannot revoke these freedoms as long as you follow these license terms:

- **Attribution** - You must give appropriate credit (mentioning that your work is derived from work that is Copyright (c) CodeRefinery and, where practical, linking to <https://coderefinery.org> [↗], provide a [link to the license](#) [↗], and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

No additional restrictions - You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits. With the understanding that: